

Informationsvisualisierung – Übung 10: Implizite Baumdarstellungen (Treemaps)

Lauffähiger Code

<https://github.com/yaniktf/iv-repo>

Ausführung

```
cd 10
elm reactor
```

Anschließend im Browser `http://localhost:8000` öffnen und dort `Main.elm` auswählen.

Datenstruktur und Laden

Als Datenstruktur wird wie vorgegeben die Baum-Implementierung `zwiliias/elm-rosetree` (`Tree`) genutzt. Jeder Knoten trägt ein Paar `( String, Int )` aus Name und Wert. Die Datei `flare.json` wird beim Start per `Http.get` geladen; ein rekursiver Decoder überführt das geschachtelte JSON in einen `Tree ( String, Maybe Int )`:

```
treeDecoder : Decoder (Tree ( String, Maybe Int ))
treeDecoder =
  Decode.map3
    (\name value children ->
      Tree.tree ( name, value ) (Maybe.withDefault [] children))
    (Decode.field "name" Decode.string)
    (Decode.maybe (Decode.field "value" Decode.int))
    (Decode.maybe
      (Decode.field "children"
        (Decode.list (Decode.lazy (\_ -> treeDecoder))))))
```

`Decode.lazy` ist nötig, da der Decoder sich für die Kinder selbst aufruft. Nach dem Laden wird der Wert der Blätter über `computeSums` rekursiv in die inneren Knoten aufaddiert, sodass jeder innere Knoten die Summe seiner Kinder als Fläche kennt:

```
computeSums : Tree ( String, Int ) -> Tree ( String, Int )
computeSums tree =
  let
    children = Tree.children tree |> List.map computeSums
    ownValue = Tree.label tree |> Tuple.second
    childSum =
      children
        |> List.map (Tree.label >> Tuple.second)
        |> List.sum
  in
    Tree.tree ( Tree.label tree |> Tuple.first, ownValue + childSum ) children
```

Aufgabe 10.1: Einfache Treemap (selbst implementiert)

Die einfache Treemap wird vollständig selbst gezeichnet. Die Achsen werden abwechselnd als Split-Achse genutzt, beginnend mit der x-Achse. Um die Aufteilung der Kinder eines Knotens zu berechnen, wird der verfügbare Platz entlang der Split-Achse linear auf die Summe der Kinder-Werte skaliert. Jedes Kind erhält dadurch einen Anteil $\text{childValue} / \text{childTotal}$ der Länge, wird als Rechteck eingezeichnet und `drawTreeNode` ruft sich für dessen Kinder rekursiv mit umgekehrter Split-Achse (`not splitX`) auf:

```
drawTreeNode : Bool -> Float -> Float -> Float -> Float
              -> Tree ( String, Int ) -> List (Svg Msg)
drawTreeNode splitX x y width height tree =
  ... children
    |> List.foldl
      (\child ( offset, acc ) ->
        let
          childSize = childValue / childTotal
          nextWidth  = if splitX then width * childSize else width
          nextHeight = if splitX then height else height * childSize
          nextX      = if splitX then offset else x
          nextY      = if splitX then y else offset
          nextOffset =
            if splitX then offset + nextWidth
            else offset + nextHeight
        in
          ( nextOffset
            , acc ++ drawTreeNode (not splitX)
                          nextX nextY nextWidth nextHeight child )
      ) ( if splitX then x else y, [] )
```

Über die `foldl`-Akkumulation wandert der `offset` entlang der aktuellen Split-Achse weiter, sodass die Kinder lückenlos nebeneinander (bzw. übereinander) liegen. Die Treemap wird zunächst für den kleinen `testTree` und anschließend für die Flare-Daten gezeichnet.

Aufgabe 10.2: Squarified Treemap (Hierarchy.treemap)

Für die Squarified Treemap wird die Funktion `Hierarchy.treemap` aus dem Modul `Hierarchy` der Bibliothek `gampleman/elm-visualization` genutzt. Über `Hierarchy.tile` `Hierarchy.squarify` entstehen möglichst quadratische Kacheln; `Hierarchy.size` legt die Zeichenfläche fest und der Zugriffsfunktion (`Tuple.second >> toFloat`) entnimmt sie den Flächenwert je Knoten:

```
positionedTree =
  Hierarchy.treemap
    [ Hierarchy.size canvasWidth canvasHeight
    , Hierarchy.paddingInner (\_ -> 1)
    , Hierarchy.paddingOuter (\_ -> 0)
    , Hierarchy.tile Hierarchy.squarify
    ]
  (Tuple.second >> toFloat)
  tree
```

Der so positionierte Baum (`Tree PositionedNode` mit `x/y/width/height`) wird mit `Tree.depthFirstFold` in SVG-Rechtecke übersetzt. Labels werden nur gezeichnet, wenn die Kachel groß genug ist (`labelIfRoom`).

Färbung ab Ebene 1 und ab Ebene 2

Im Beispiel sind Teilbäume ab einer Ebene unter der Wurzel verschieden gefärbt. Zusätzlich wird gefordert, Teilbäume ab der *zweiten* Ebene unter der Wurzel verschieden zu färben. Beide Modi werden über einen Typ `ColorMode` getrennt. Die Farbe ergibt sich aus dem Pfad-Element auf der gewählten Ebene: Bei `ColorByLevel1` entscheidet das Element an Index 1 (direktes Kind der Wurzel), bei `ColorByLevel2` das an Index 2 – alle Knoten innerhalb desselben Teilbaums teilen sich dadurch eine Farbe:

```
type ColorMode = ColorByLevel1 | ColorByLevel2

colorClassForNode mode ancestors node =
  let
    path = List.reverse (node :: ancestors)
              |> List.map (.node >> Tuple.first)
    key =
      case mode of
        ColorByLevel1 ->
          itemAt 1 path |> Maybe.withDefault "root"
        ColorByLevel2 ->
          itemAt 2 path
              |> Maybe.withDefault
                (itemAt 1 path |> Maybe.withDefault "root")
  in
    "color-" ++ String.fromInt (paletteIndex key)
```

Der `key` wird über `hashString` stabil auf einen von zwölf Palettenplätzen (`modBy 12`) abgebildet, so dass gleiche Teilbäume reproduzierbar dieselbe Farbe erhalten. Bei `ColorByLevel2` fällt ein Knoten, der noch keine zweite Ebene besitzt, auf seine Ebene-1-Farbe zurück.

Überblick

Aufgabe	Verfahren	Layout
10.1 einfache Treemap	selbst, x/y alternierend	lineare Skalierung
10.2 squarified Treemap	<code>Hierarchy.treemap</code>	<code>squarify</code> , Farben Ebene 1 & 2